Tabăra de pregătire a lotului național de informatică juniori

Ploiești, 23-28 aprilie 2026

Descrierea soluțiilor Baraj 2

Comisia științifică

27 aprilie 2026

Problema 1. ERA

Propusă de: prof. Ciprian Cheșcă, Liceul Tehnologic Grigore Moisil Buzău
prof. Ionel-Vasile Piț-Rada, Colegiul Național Traian, Drobeta Turnu Severin

Soluție – prof. Ciprian Cheșcă – backtracking în plan

Pentru a determina numărul de puncte laticiale (de coordonate întregi) care pot fi atinse de capătul liber al ERA putem aplica metoda **backtracking**.

Codificăm cele patru direcții posibile în care poate fi așezat un segment astfel:

- 1 - pentru așezarea unui segment pe orizontală cu capătul inițial în stânga și cel final către dreapta.
- 2 - pentru așezarea unui segment pe verticală cu capătul inițial în jos și cel final mai sus.
- 3 - pentru așezarea unui segment pe orizontală cu capătul inițial în dreapta și cel final către stânga.
- 4 - pentru așezarea unui segment pe verticală cu capătul inițial sus și cel final mai jos.

Putem înțelege mai bine aceste codificări asociindu-le cele patru puncte cardinale: 1-EST, 2-NORD, 3-VEST și 4-SUD.

Generăm apoi pe stivă numerele 1, 2, 3, 4 și pentru fiecare configurație calculăm ce poziție în plan corespunde acestei configurații, adică calculăm o abscisă și o ordonată. Apoi marcăm într-o matrice această poziție ca fiind vizitată.

În final parcurgem matricea și numărăm pozițiile marcate.

Această soluție are complexitate $O(4^N)$.

Optimizare

Din simetria așezării punctelor ce respectă cerința problemei se poate observa că aceste puncte formează întotdeauna o figură care este formată dintr-o linie oblică inițială (cea care este cea mai depărtată de origine) care se repetă de exact de atâtea ori câte puncte se găsesc pe această linie. Din această cauză numărul căutat este un **pătrat perfect** și este suficient să aflăm câte puncte sunt pe această linie. Punctele de pe această linie se obțin doar prin așezare spre EST sau NORD a segmentelor deoarece sunt cele mai depărtate de origine și nu conțin întoarceri, așadar este suficient să generăm pe stivă doar 1 sau 2. Soluția are complexitate $O(2^N)$.

Subtask 2: Enumerarea tuturor punctelor

Procesăm segmentele unul câte unul. Menținem A_k = lista tuturor punctelor în care poate ajunge un braț robotic format din primele k segmente. Din A_k obținem A_{k+1}. Pentru aceasta, luăm fiecare punct din A_k și îl deplasăm în sus / jos / la stânga / dreapta cu o distanță egală cu lungimea celui de-al $k+1$-lea segment. Dorim ca A_{k+1} să nu conțină duplicate. Deoarece lungimea totală a brațului nu depășește 1 000, punctele au coordonate în intervalul $[-1\,000, +1\,000]$. Putem reține o matrice de $2\,001 \times 2\,001$ booleeni în care bifăm coordonatele deja obținute (trebuie să golim această matrice după generarea fiecărei liste, în vederea iterației următoare).

Complexitatea este $O(N \cdot (LN)^2) = O(N^3 \cdot L^2)$, deoarece generăm N liste și fiecare listă conține $O((NL)^2)$ puncte.

Pentru restul punctelor, este necesară o observație despre structura punctelor accesibile.

Demonstrație că numărul de soluții este pătrat perfect

Să studiem exemplul din enunț pentru $L = \{3, 4\}$. Observăm că el „arată” ca o rețea de 4×4 puncte aliniată cu bisectoarele axelor. Punctele nu sunt neapărat echidistante. Să denumim această structură o **grilă**. Pentru a simplifica aritmetica, să facem următoarele transformări:

1. Rotim planul cu 45° în sens trigonometric.
2. Amplificăm coordonatele cu $\sqrt{2}/2$, ceea ce face ca punctele grilei să capete coordonate întregi.
3. Punem originea în cel mai din stânga-jos punct.

Matematic, transformarea este:

$$\begin{cases} x \longrightarrow \frac{1}{2}(x - y + \Sigma_k) \\ y \longrightarrow \frac{1}{2}(x + y + \Sigma_k) \end{cases}$$

unde Σ_k este suma lungimilor primelor k segmente. Numim figura rezultată **grilă rotită**. Pentru exemplul din enunț obținem grila rotită:

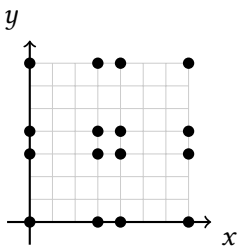

Figura 1: Grila rotită pentru exemplul din enunț.

Pe acest exemplu particular observăm că, pentru orice $x, y \in \{0, 3, 4, 7\}$, punctul (x, y) aparține grilei, iar 3 și 4 sunt exact lungimile celor două segmente. Să generalizăm această observație.

Fie G_k $(0 \leq k \leq N)$ grila rezultată după procesarea primelor k segmente. Fie D_k numărul de puncte de pe axa Ox ale grilei G_k, de coordonate $x_0, x_1, \ldots, x_{D_k-1}$.

Propoziție. Pentru orice $0 \leq i, j < D_k$, punctul de coordonate (x_i, x_j) aparține grilei.

Demonstrația este inductivă. Cazul de bază este $k = 0$, caz în care G_0 conține doar punctul $(0, 0)$. Pentru a face pasul de inducție, să presupunem că propoziția este adevarată pentru G_k și să vedem ce efect are adăugarea la G_k a segmentului $k + 1$ de lungime l. El va multiplica grila (originală) prin translatare cu $\pm l$ pe verticală sau pe orizontală. În planul rotit, efectul este multiplicarea grilei rotite prin translatare cu l la dreapta **sau** cu l în sus **sau** cu l în sus și la dreapta, raportat la originea aleasă. Pentru exemplul din enunț, dacă adăugăm $l = 15$, obținem figura de mai jos în care recunoaștem patru „subgrile" (patru copii ale lui G_k).

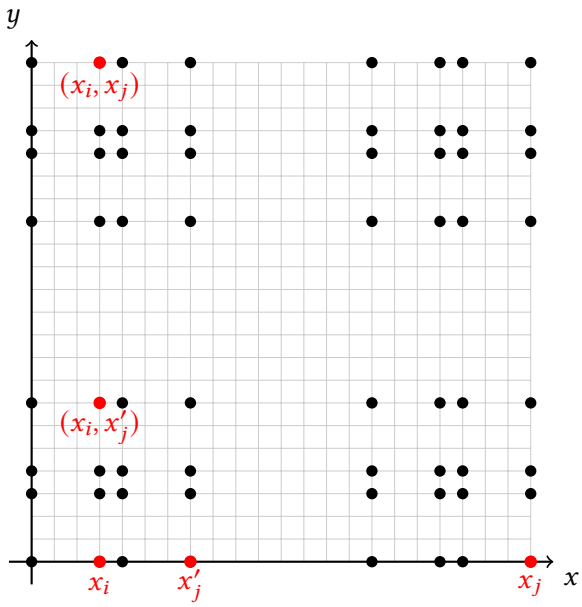

Figura 2: Obținerea lui G_{k+1} din G_k. Cu roșu sunt figurate două puncte $(x_i, 0)$, $(x_j, 0)$ și secvența de puncte care arată că (x_i, x_j) face parte din G_{k+1}

Să fixăm arbitrar două puncte $(x_i, 0), (x_j, 0) \in G_{k+1}$. Iau naștere patru cazuri:

1. Dacă $(x_i, 0), (x_j, 0) \in G_k$ atunci ne aflăm în subgrila din stînga-jos, care este o copie a lui G_k și știm că $(x_i, x_j) \in G_k \subseteq G_{k+1}$.

2. Dacă $(x_i, 0) \in G_k$ și $(x_j, 0) \notin G_k$, atunci de unde a apărut punctul $(x_j, 0)$? El este imaginea translatată cu l a unui alt punct $(x'_j, 0) \in G_k$, așadar $x_j = x'_j + l$. Știm că $(x_i, x'_j) \in G_k$ și știm că acest punct a fost deplasat și în sus pe y cu l. De aceea, $(x_i, x'_j + l) \in G_{k+1}$, adică $(x_i, x_j) \in G_{k+1}$.

3. Celelalte două cazuri sunt tratate similar cu cazul (2).

Astfel am demonstrat că numărul de puncte din G_k este D_k^2. Pentru a afla răspunsul, este suficient să aflăm valoarea D_N.

Reducere la problema rucsacului

Așadar, să numărăm punctele din grila G_k de pe axa Ox.

Vom proceda recurent. G_0 conține doar originea. După adăugarea segmentului $k + 1$ de lungime l, putem ajunge la o coordonată x:

1. dacă puteam ajunge la acea coordonată și după k segmente;

2. dacă puteam ajunge la coordonata $x - l$ după k segmente.

Privind în ansamblu, o coordonată x este accesibilă dacă poate fi exprimată ca sumă a lungimilor unei submulțimi S de segmente. Semnificația este că pentru segmentele din S urmăm copia grilei prin deplasarea la dreapta, iar pentru celelalte segmente urmăm copia originală a grilei (cea care rămâne cu colțul în origine).

Așadar, am redus calculul lui D_N la întrebarea: câte sume distincte se pot obține cu toate submulțimile celor N segmente? Aceasta este o problemă clasică, numită *Rucsac 0/1*.

Fără nicio optimizare, deoarece avem N obiecte de adăugat în rucsac, iar rucsacul are capacitate NL, complexitatea este $O(N^2L)$.

Optimizări

Pentru a obține 100 puncte se poate folosi o tehnică prin care, dacă în set unele valori se repetă de multe ori, atunci se poate înlocui setul de N lungimi dat cu un set nou, cu mai puține numere, care va produce aceeași plajă de valori distincte, dar în care avem un număr de valori aproximativ egal cu $100 \cdot \log(N/100)$, deci mult mai puține valori.

Optimizarea I: Descompunerea în puteri ale lui 2

Dacă avem multe segmente de aceeași lungime (de exemplu, lungimea 5 apare de 100 de ori), procesarea lor unul câte unul este ineficientă ($O(N)$).

Am folosit descompunerea binară: Orice număr X poate fi descompus în puteri ale lui 2 (1, 2, 4, 8 , ...). Astfel, cele 100 de segmente de lungime 5 sunt grupate în „pachete” de dimensiuni: 1×5, 2×5, 4×5, 8×5, 16×5, 32×5 și un rest de 37×5.

- În loc de 100 de operații pe bitset, facem doar 7.
- Această metodă reduce complexitatea de la $O(N)$ la $O(\log N)$ pentru fiecare lungime distinctă. Avem în continuare un rucsac de capacitate NL, în care introducem $L \log N$ obiecte, deci complexitatea totală devine $O(NL^2 \log N)$.

Optimizarea II: Performanță la nivel de procesor (0,027s)
Pentru a coborî sub limita de 0,1s, am implementat câteva optimizări critice:

- Bitset Manual pe 64 biți: În loc de `std::bitset`, am folosit un tablou de numere `uint64_t`. Aceasta permite procesorului să lucreze cu „cuvinte" de 64 de biți nativi, eliminând verificările inutile ale bibliotecilor standard.
- Fast I/O (`fread`): Citirea a 100 000 de numere cu `cin` sau `scanf` este lentă. Am utilizat un parser propriu care citește blocuri mari de date direct din fișier în memorie, reducând timpul de citire la aproape zero.
- Popcount Hardware: Pentru numărarea finală a biților de 1 (sumele posibile), am folosit funcția `__builtin_popcountll`. Aceasta apelează o instrucțiune specială a procesorului care numără biții dintr-un registru într-un singur ciclu de ceas.
- Limitarea intervalului: Nu am procesat niciodată întregul bitset de 10 milioane de poziții, ci doar până la suma maximă atinsă în acel moment, economisind timp prețios.

O soluție în $O(NL^2)$

Pentru o optimizare asimptotică, să considerăm ce ar face rucsacul „brut" pentru $k = 5$ obiecte de lungime $l = 3$, la coordonata 50. Am obține un 1 (adică este posibil să ajungem la coordonata 50) dacă, înainte să procesăm lungimea 3, aveam un 1 pe oricare dintre pozițiile 50, 47, 44, 41, 38, 35. Aceasta înseamnă că putem procesa **toate** segmentele de lungime l **într-o singură trecere** prin rucsac. Trebuie ca, atunci când procesăm lungimea l cu frecvența $f(l)$ și baleiem rucsacul, să menținem suma ultimelor $f(l) + 1$ poziții din l în l. Complexitatea devine $O(NL^2)$.

Această implementare are șanse să ia 100p, dar ea este încetinită de faptul că nu putem opera pe bitseturi, deoarece trebuie să accesăm pozițiile din rucsac (coordonatele) una câte una ca să menținem sumele sus-menționate. Este preferabilă o implementare care profită de operațiile în masă specifice bitseturilor, în particular shiftarea la stânga și OR-ul pe biți.

De aceea, descompunerea binară descrisă anterior se comportă mai bine în practică. Ea are complexitatea $O(NL^2 \log N)$, asimptotic mai slabă decât soluția în $O(NL^2)$. Dar constanta foarte bună de 1/64 și amprenta mai mică de memorie o fac superioară.

Soluție – prof. Ionel Vasile Piț-Rada – număr de submulțimi de sume distincte

Pentru completitudine, includem și o demonstrație completă din punct de vedere matematic pentru determinarea numărului de puncte, mai riguroasă decât cea anterioară, care este mai vizuală.

Să observăm că dacă un segment este așezat vertical lungimea lui nu va influența valoarea coordonatei finale pe Ox, dar va influența valoarea coordonatei finale pe Oy și reciproc dacă segmentul este așezat orizontal. Deci pentru fiecare segment este suficient să asociem o stare 0 (vertical) sau 1 (orizontal). Parcurgând acest șir de biți și adunând lungimile segmentelor carea au starea asociată egală cu 1 la coordonata x și respectiv la coordonata y pe cele cu starea asociată egală cu 0, obținem cele două coordonate ale capătului brațului. Este acum destul de clar că avem nevoie de D, numărul sumelor distincte obținute pe una din coordonate, acest număr fiind același și pentru cealaltă coordonată, astfel $D \cdot D$ va da soluția căutată.

Să demonstrăm acum că **numărul** P de puncte cu coordonate întregi în care poate ajunge capătul liber al ultimului segment al ERA **este egal** cu **pătratul numărului** D **de sume distincte de submulțimi ale mulțimii lungimilor segmentelor** $\{L_1, L_2, \ldots, L_N\}$**. Adică** $P = D^2$.

Notații și cadru

Avem N segmente cu lungimi naturale nenule $L_1, L_2, \ldots, L_N$. Primul segment S_1 este fixat cu un capăt în origine (0,0), iar fiecare segment poate fi orientat paralel cu axele Ox sau Oy, în oricare dintre cele două sensuri. Notăm $T = L_1 + L_2 + \cdots + L_N$. Notăm Σ setul tuturor sumelor de submulțimi ale $\{L_1, \ldots, L_N\}$, incluzând 0 (submulțimea vidă). Notăm $D = |\Sigma|$ (numărul de sume distincte).

Afirmația de demonstrat

Numărul P al pozițiilor accesibile (x, y) cu coordonate întregi pentru capătul liber al ultimului segment este $P = D^2$.

Construcția bijecției $(\Sigma \times \Sigma) \leftrightarrow$ (poziții accesibile)

Considerăm două submulțimi $P, Q \subseteq \{1, 2, \ldots, N\}$.

Notăm $a = \sum_{i \in P} L_i$ și $b = \sum_{i \in Q} L_i$, deci $a, b \in \Sigma$. Vom arăta cum, din perechea (a, b), obținem o configurare a segmentelor care plasează vârful în punctul (x, y) determinat prin formulele $x = a + b - T$ și $y = a - b$.

Regula de orientare a fiecărui segment i este următoarea:

- dacă $i \in P \cap Q$, orientăm segmentul i orizontal spre $+x$;
- dacă $i \in P \setminus Q$, orientăm segmentul i vertical spre $+y$;
- dacă $i \in Q \setminus P$, orientăm segmentul i vertical spre $-y$;
- dacă $i \notin P \cup Q$, orientăm segmentul $iorizontalspre - x$.

Suma contribuțiilor pe axa Ox este $+x$ pentru $P \cap Q$ și $-x$ pentru $\overline{P \cup Q}$, deci $x = (\sum_{i \in P \cap Q} L_i) - (\sum_{i \notin P \cup Q} L_i) = a + b - T$.

Analog, suma contribuțiilor pe axa Oy este $+y$ pentru $P \setminus Q$ și $-y$ pentru $Q \setminus P$, deci $y = (\sum_{i \in P \setminus Q} L_i) - (\sum_{i \in Q \setminus P} L_i) = a - b$.

(Injecție:) perechi diferite (a, b) dau puncte diferite (x, y)

Dacă $(a, b) \neq (a', b')$ în $\Sigma \times \Sigma$, atunci cel puțin una dintre coordonatele $x = a + b - T$ sau $y = a - b$ diferă de $x' = a' + b' - T$, respectiv $y' = a' - b'$. Prin urmare, punctele rezultate sunt diferite. (Avem astfel o aplicație injectivă $\Sigma \times \Sigma \to$ poziții accesibile).

(Surjecție:) orice punct accesibil provine dintr-o pereche $(a, b) \in \Sigma \times \Sigma$

Fie un punct accesibil (x, y), obținut printr-o anumită orientare a segmentelor. Grupăm indicii segmentelor în patru categorii, corespunzătoare celor patru direcții posibile: $+x, -x, +y, -y$. Notăm P mulțimea indicilor orientați fie spre $+x$, fie spre $+y$, iar Q mulțimea indicilor orientați fie spre $+x$, fie spre $-y$, astfel încât:

- i în $+x \implies i \in P \cap Q$;
- i în $+y \implies i \in P \setminus Q$;
- i în $-y \implies i \in Q \setminus P$;
- i în $-x \implies i \in \overline{P \cup Q}$.

Definind $a = \sum_{i \in P} L_i$ și $b = \sum_{i \in Q} L_i$, se verifică direct că (x, y) satisfac $x = a + b - T$ și $y = a - b$, cu $a, b \in \Sigma$. Rezultă că orice punct accesibil este imaginea unei perechi $(a, b) \in \Sigma \times \Sigma$; (deci aplicația este surjectivă).

Concluzie

(Am construit o bijecție între $\Sigma \times \Sigma$ și mulțimea pozițiilor accesibile). Prin urmare, numărul de poziții este $|\Sigma \times \Sigma| = |\Sigma|^2 = D^2$. Deci $P = D^2$, unde D este numărul de sume distincte de submulțimi ale $\{L_1, \ldots, L_N\}$.

Exemple de verificare

1. $N = 2$, $L = \{3, 4\}$. Sumele de submulțimi sunt $\{0, 3, 4, 7\}$, deci $D = 4$. Rezultă $P = D^2 = 16$, conform figurii din enunț.
2. $N = 8$, $L = \{50, 47, 23, 35, 10, 49, 5, 4\}$. Dacă $P = 28\,224$, atunci $D = \sqrt{P} = 168$. Rezultă că există 168 sume distincte ale submulțimilor, iar $P = 168^2$.

Observație

Rezultatul este independent de ordinea segmentelor, numai mulțimea lungimilor contează.

Cazurile în care $P = (sum + 1)^2$

Se notează $sum = L_1 + L_2 + \cdots + L_N$ (suma lungimilor segmentelor). Rezultatul general pentru problema ERA este $P = D^2$, unde D este numărul sumelor distincte de submulțimi. Ne interesează situațiile în care $P = (sum + 1)^2$.

Criteriul necesar și suficient

Rezultatul este valabil exact atunci când sumele submulțimilor acoperă întreg intervalul $[0, sum]$, adică există $D = sum + 1$ sume distincte (toate valorile posibile).

O caracterizare echivalentă, clasică în teoria sumelor de submulțimi, este următoarea:

1. Ordonăm crescător lungimile $a_1 \leq a_2 \leq \cdots \leq a_N$.
   Definim $r = 0$ (cea mai mare valoare până la care putem construi toate sumele).

2. Parcurgem segmentele în ordine:

   - dacă pentru fiecare k avem $a_k \leq r + 1$, atunci la final obținem $r = sum$ și toate sumele $0 \ldots sum$ sunt posibile;
   - dacă pentru un k apare $a_k > r + 1$, există o „gaură” în acoperirea intervalului și nu toate sumele sunt realizabile.

Concluzie

Astfel, $P = (sum + 1)^2$ dacă și numai dacă, după sortarea lungimilor, pentru fiecare k se verifică:

$$a_k \leq 1 + (a_1 + a_2 + \cdots + a_{k-1})$$

adică fiecare segment nou este cel mult cu 1 mai mare decât suma precedentă. În această situație, mulțimea sumelor de submulțimi este exact $[0, sum]$.

Exemple

1. $L = \{1, 2, 3\}$. Avem $sum = 6$. Se pot obține toate sumele din $[0, 6]$, deci $D = 7$, $P = 7^2 = 49 = (6 + 1)^2$.

2. $L = \{1, 1, 1\}$. Avem $sum = 3$. Toate sumele $0 \ldots 3$ sunt posibile, deci $P = (3 + 1)^2 = 16$.

3. $L = \{3, 4\}$. $sum = 7$. Lipsesc sumele intermediare (ex. 1, 2, 5, 6), deci $D < 8$ și $P \neq (7 + 1)^2$.

4. $L = \{1, 2, 4, 8\}$. $sum = 15$. Puterile lui 2 acoperă tot intervalul $[0, 15]$, deci $P = (15 + 1)^2 = 256$.

Problema 2. Jucării

Propusă de: instr. Cătălin Frâncu, Nerdvana 2.0 București

Soluție elementară ($M \leq 4$) (5 puncte)

Cînd $M = 3$, există doar trei moduri de a organiza jucăriile: grupăm oricare două dintre jucării, iar a treia stă singură. Pentru fiecare mod, trecem prin vectorul J în $O(N)$ și numărăm deschiderile dulapului. Complexitatea este $O(N)$.

Similar putem proceda și pentru $M = 4$, unde există tot trei organizări: 1 grupată cu 2, cu 3, respectiv cu 4 și celelalte două jucării între ele.

Soluții bazate pe backtracking (12-51 de puncte)

Putem să generăm toate permutările posibile ale mulțimii $\{1, 2, \ldots, M\}$. Fiecare permutare semnifică exact că grupăm prima jucărie cu a doua, a treia cu a patra etc. Pentru fiecare permutare, trecem în $O(N)$ prin vectorul de jucării și calculăm numărul de deschideri ale dulapului. Fără optimizări, această implementare trece primele două grupe de teste. Complexitatea ei este $O(M! \cdot N)$.

O optimizare majoră, care ne ajută la toate soluțiile, este să observăm că, pentru o așezare a jucăriilor, numărul de deschideri ale dulapului este N minus numărul de perechi consecutive (dintre cele $N-1$) care stau în aceeași cutie. De aceea, putem să reducem vectorul J la o matrice A de mărime $M \times M$ unde $A_{i,j}$ denotă **afinitatea** jucăriilor i și j, adică numărul de perechi (i, j) sau (j, i) de la intrare. Dacă J conține și perechi de jucării egale (i, i), scădem din răspuns numărul acestor perechi, căci a doua apariție în astfel de perechi întotdeauna va găsi cutia deja scoasă din dulap. Cu această reprezentare, definim **scorul** unei împerecheri ca suma afinităților perechilor. Atunci răspunsul la problemă este N, minus scorul împerecherii optime, minus numărul de perechi de jucării consecutive egale din J.

Cu această optimizare, putem reduce complexitatea backtrackingului la $O(M!)$: pe măsură ce generăm permutarea, îi calculăm incremental și scorul. Astfel, ori de cîte ori backtrackingul ajunge pe nivelul M știm răspunsul pentru acea permutare. Această soluție rezolvă și grupa 3 și obține 23 de puncte.

Alte optimizări încearcă să evite redundanța permutărilor, care de exemplu vor încerca să pună jucăriile 1 și 2 în prima cutie, apoi în a doua, în a treia etc. Toate aceste așezări sînt echivalente. O abordare care pur și simplu pune jucăria 1 în prima cutie (sau, dacă M este impar, încearcă și scenariul în care jucăria 1 rămîne neîmperecheată) va trece și grupa a patra și va obține 36 de puncte. Mai general, putem ca pe prima poziție din fiecare cutie să punem doar jucăria cu indicele minim disponibilă. Doar pe a doua poziție din fiecare cutie încercăm toate posibilitățile. Acest algoritm are complexitatea $O(1 \cdot 3 \cdot 5 \cdot \cdots \cdot M)$ și trece primele 5 grupe de teste (51 de puncte).

Operarea pe măști de biți poate fi un instrument util. În plus față de un simplu vector caracteristic, care ne spune dacă o jucărie este disponibilă sau nu, măștile de biți ne pot oferi în $O(1)$ indicele primei jucării disponibile, cu funcția `__builtin_ctz(masca)`.

În sfîrșit, menționăm o implementare de backtracking care trece toate testele, chiar în timp mai bun decît soluția de 100p. Ea precalculează, pentru fiecare jucărie i, lista ordonată a celorlalte jucării j în ordine descrescătoare a afinității. Apoi, la fiecare moment în procesul de generare a permutărilor, algoritmul se întreabă: dacă fiecare jucărie dintre cele rămase și-ar putea alege perechea favorită tot dintre cele rămase, ce scor am obține? Ignorăm faptul că unele elemente vor fi împerecheate de mai multe ori, deci estimarea scorului va fi optimistă. Dacă, în aceste condiții, estimarea nu depășește scorul maxim deja găsit pînă acum, putem părăsi nivelul curent. Acest backtracking, foarte bine informat, taie devreme variantele care nu au șanse de reușită (împerecheri cu afinitate foarte mică etc.).

Soluții greedy (0-50 de puncte)

În principiu, testele nu sînt ușor de trecut cu soluții greedy. De exemplu, un greedy natural este: cuplează jucăriile a și b care maximizează $A_{a,b}$. Dintre cele $M-2$ jucării rămase, cuplează jucăriile c și d care maximizează $A_{c,d}$, etc. Acesta nu va lua puncte. Chiar și în exemplul din enunț, cea mai profitabilă pereche este 3-7, dar soluția optimă folosește perechile 3-5 și 7-8.

O abordare mult mai eficientă este **greedy randomizat**. În mod repetat, acest greedy generează o permutare aleatorie și îi calculează scorul. La final, raportează permutarea de scor maxim găsită. El este foarte eficient, rulînd în $O(M)$ per permutare, deci poate face sute de mii de experimente. În implementările noastre, el obține pînă la 46 de puncte.

Soluție cu programare dinamică pe măști de biți (100 de puncte)

Vom calcula scorul fiecărei submulțimi de jucării, dacă am împerechea doar jucăriile din acea submulțime. Atunci pentru o submulțime S iau naștere două cazuri:

- Dacă $|S|$ este impar, una dintre jucăriile din S va rămîne neîmperecheată. Încercăm pe rînd să eliminăm cîte un element din S, iar scorul lui S este maximul dintre scorurile submulțimilor rezultate.
- Dacă $|S|$ este par, fixăm una dintre jucăriile din S (de exemplu, pe cea de indice minim), să-i spunem i. Această jucărie trebuie împerecheată cu o altă jucărie, să-i spunem j. Încercăm toate valorile posibile pentru $j \in S \setminus \{i\}$. Scorul lui S este maximul valorilor $\{A_{i,j} + scor(S \setminus \{i, j\})\}$

Pentru o implementare eficientă, îi asociem fiecărei submulțimi de jucării o mască pe M biți în care bitul i este 1 dacă și numai dacă jucăria i este în submulțime. Aceasta ne permite o reprezentare compactă a tuturor scorurilor folosind un vector de 2^M elemente. Pentru reconstituirea soluției, în fiecare dintre cele două cazuri de mai sus trebuie să reținem care dintre submulțimile lui S a generat scorul optim:

- Dacă $|S|$ este impar, reținem jucăria care rămîne neîmperecheată.
- Dacă $|S|$ este par, reținem jucăria cu care o împerechem pe i.

Observăm că aflarea scorului lui S depinde de cunoașterea scorului anumitor submulțimi ale lui S. Așadar, submulțimile trebuie considerate într-o anumită ordine. Este suficient să considerăm măștile de la 0 la $2^M - 1$ în această ordine.

Complexitatea acestui algoritm este $O(M \cdot 2^M + N)$, deoarece avem de analizat 2^M submulțimi și pentru fiecare submulțime iterăm prin obiectele din submulțime.

Problema 3. Sport

Propusă de: prof. Bogdan-Ioan Popa, Liceul Teoretic Just4Kids București
prof. Daniel Popa, Colegiul Național Aurel Vlaicu Orăștie

Idee de rezolvare

Observația 1

Observăm că este suficient ca un elev să fie mutat cel mult o dată. Atunci vrem să maximizăm numărul de elevi păstrați în ordinea lor relativă inițială. Răspunsul este $N -$ max_păstrat, unde max_păstrat este numărul maxim de elevi care pot rămâne în ordinea lor relativă inițială, fără a fi mutați.

Observația 2

Un șir de înălțimi $H_1, H_2, \ldots, H_N$ permite profesorului să formeze linia în ordine crescătoare (plasând fiecare elev la față sau la spate) dacă și numai dacă:

- **Cazul 1**:
  - Elevii cu înălțimea $\leq H_1$ apar în ordine **descrescătoare** în șir;
  - Elevii cu înălțimea $> H_1$ apar în ordine **crescătoare** în șir.
- **Cazul 2**:
  - Elevii cu înălțimea $< H_1$ apar în ordine **descrescătoare** în șir;
  - Elevii cu înălțimea $\geq H_1$ apar în ordine **crescătoare** în șir.

Reducerea problemei

Dorim cel mai lung subșir al secvenței originale care satisface condiția de mai sus. Răspunsul este N minus lungimea acestui subșir.

Pentru a determina acest subșir, putem încerca fiecare elev ca punct de referință H_1 și testează condițiile pentru a găsi cel mai lung subșir valid. Vom calcula dp_low_i și dp_high_i pentru fiecare elev i, unde:

- dp_low_i este lungimea celui mai lung subșir descrescător care începe la poziția i;
- dp_high_i este lungimea celui mai lung subșir crescător care începe la poziția i.

Relațiile de recurență pentru aceste valori sunt:

- $dp_low_i = 1 + \max(dp_low_j)$ pentru toate $j > i$ și $H_j \leq H_i$;
- $dp_high_i = 1 + \max(dp_high_j)$ pentru toate $j > i$ și $H_j \geq H_i$.

Vom fixa un elev i ca punct de referință (ca și cum ar fi H_1) și vom testa condițiile pentru a determina lungimea subșirului de lungime maximă valid care poate fi format în jurul acestui elev.

- **Cazul 1**:
  - Se determină j pentru care $H_j \leq H_i$ și dp_low_j este maxim, iar $i < j$;
  - Se determină j pentru care $H_j > H_i$ și dp_high_j este maxim, iar $i < j$;
- **Cazul 2**:
  - Se determină j pentru care $H_j < H_i$ și dp_low_j este maxim, iar $i < j$;
  - Se determină j pentru care $H_j \geq H_i$ și dp_high_j este maxim, iar $i < j$;

  Rezultatul va fi $1 + dp_low_j + dp_high_j$.

Vom alege cel mai mare rezultat obținut pentru toți elevii i și vom scădea acest rezultat din N pentru a obține răspunsul final.

În funcție de modalitatea de calculare a recurențelor, respectiv combinarea rezultatelor din cele două cazuri, putem obține un algoritm de complexitate $O(N^2)$ prin implementarea directă, sau putem optimiza calculul acestor recurențe folosind arbori de intervale, arbori indexați binar sau căutare binară pentru a obține complexitatea optimă de $O(N \log N)$.

Recomandări de lectură:

- pbinfo.ro
- infoarena.ro

Echipa

Problemele pentru această etapă au fost pregătite de:

- prof. Emanuela Cerchez, Colegiul Național „Emil Racoviță” Iași
- prof. Ionel-Vasile Pit-Rada, Colegiul Național Traian, Drobeta Turnu Severin
- șef lucrări dr. Mihai Nan, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București
- prof. Dorin-Mircea Rotar, Colegiul Național „Samuil Vulcan” Beiuș
- prof. Mihai Bunget, Colegiul Național „Tudor Vladimirescu” Târgu-Jiu
- prof. Ciprian Chesică, Liceul Tehnologic Grigore Moisil Buzău
- prof. Gheorghe-Eugen Nodea, Centrul Județean de Excelență Gorj
- prof. Petru Simion Oprița, Liceul Regina Maria Dorohoi
- prof. Zoltan Szabo, Inspectoratul Școlar Județean Mureș
- prof. Marius Nicoli, Colegiul Național Frații Buzești Craiova
- prof. Dana Lica, Centrul Județean de Excelență Prahova

- instr. Cătălin Frâncu, Nerdvana 2.0 București
- stud. Aureliu Valentin Antonie, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București
- prof. Bogdan-Ioan Popa, Liceul Teoretic Just4Kids București
- stud. Alin Gabriel Răileanu, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Cristian Luchian, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Andrei Boacă, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Rareș-Andrei Cotoi, Facultatea de Matematică și Informatică, Universitatea „Babeș-Bolyai” Cluj-Napoca
- stud. Lucian-Andrei Badea, TU Delft
- stud. Ioan-Cristian Pop, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București